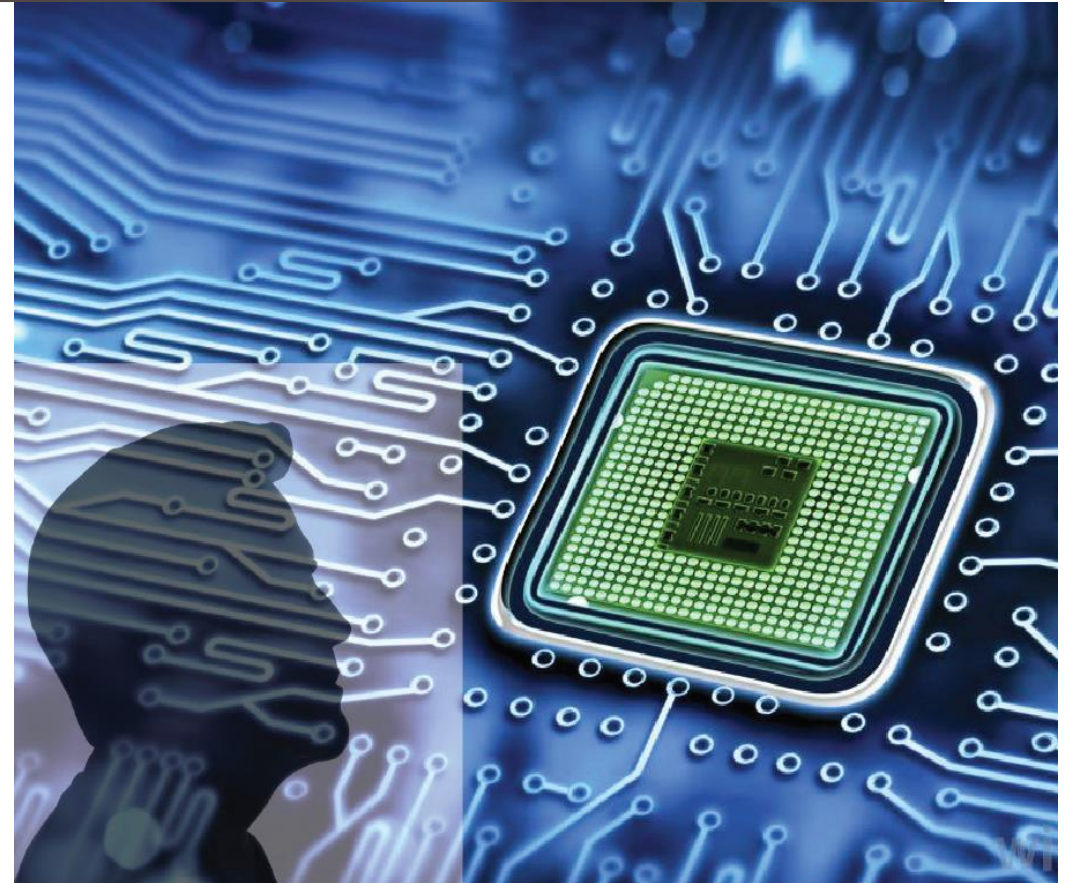


# VLSI DESIGN FLOW

## Week-5 Lecture-1 Logic Design

Sneh Saurabh  
9<sup>th</sup> September, 2024



# VDF: Revision/Announcements

---

## Attendance Policy:

- Earlier announced “student with less than 30% attendance ....”
- It will not be enforced



# Code Coverage vs. Functional Coverage

---

## Code coverage:

- Measures coverage of an existing code
- An empty module will have a 100% code coverage, while complete functionality is missing

## Functional coverage:

- Measures whether the specified design features have been exercised during simulation
- We need to provide design features using coverage model
  - Not automatically inferred by the tool
- Coverage model is independent of the design implementation.
  - Can detect missing features also in the implementation

---

---

```
graph TD; Simulation[Simulation] --> Simulator[Simulator];
```

Simulation

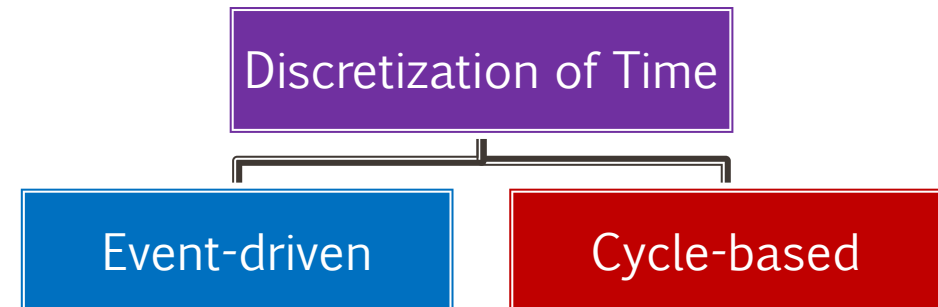
Simulator

# Simulation for Digital Circuits

| Simulation                                                                                              | Real world                                                                                         |
|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Works on abstract model                                                                                 | Works on real circuit                                                                              |
| Produces results by running simulation algorithm on a general-purpose computer (typically sequentially) | Produces results by modulation of current/voltage signals and circuit elements running in parallel |
| Expected to be slower                                                                                   | Expected to be faster                                                                              |

## Increased efficiency in simulation:

- Work at a higher level of abstraction: two-or four-valued logic
- Discretization of time: need not produce output for all values of time



# Event-driven vs. Cycle-based

---

## Event-driven Simulation:

- **Event:** when an input changes, we consider it as an event
- An event at a logic gate/signal can change its output/driven signal
  - This output/signal change can further trigger new events
- Simulation involves computing a chain of events and their responses until no further events are available

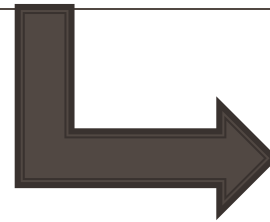
## Cycle-based Simulation (valid for synchronous circuit):

- Compute only the evolution of the **states** in a circuit.
  - output of the flip-flops and the primary output ports of a circuit are the state elements
- Evaluates the sequence of next states at successive clock edges as a function of the present state and the present input.
  - can avoid computing responses of the internal combinational logic gates

Simulators typically employ a combination of event-driven and cycle-based simulations to improve runtime.

---

# Simulation in Verilog



Mechanism

# Verilog Simulation: Definitions (1)

Verilog considers a design as consisting of connected processes.

**Processes:** design objects that a simulator can evaluate and produce a response to a given stimulus.

Examples: gate primitives, initial block, always block, continuous assignment, and procedural assignment statements.

- **Event:** Anything that requires simulator to take some action.
- Events are of two types:
  1. **Update Event:** change in the value of a net or a variable.
  2. **Evaluation Event:** evaluation of processes when an update event has occurred

```
module top(in1, in2, out1, out2);  
  input in1, in2;  
  output out1, out2;  
  reg out1, tt;  
  
  and A1(out2, tt, in1); // gate primitive  
  
  assign out1 = tt; // continuous assignment  
  
  always @(in1 or in2) // always block  
  begin  
      tt = in1 & in2;  
  end  
endmodule
```

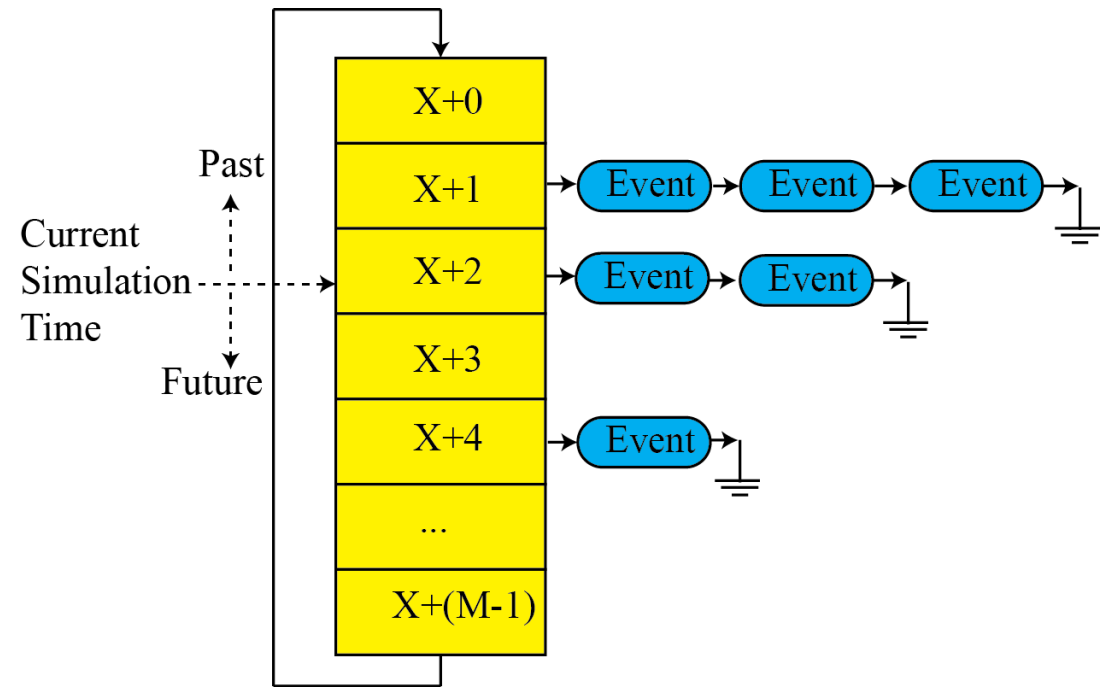
- Processes: gate instance, continuous assignment, always block
- Assume: *in1* makes a 0→1 transition
  - Update event: for the net *in1*
  - Evaluation event: in gate instance and always block (because sensitive)



# Verilog Simulation: Definitions (2)

- **Simulation time:** time value maintained by a simulator to model the actual time in the simulated circuit.
  - simulation time is different from the time taken by the simulator to simulate.
  - simulation time depends on test-bench and the design being simulated

- Events have simulation time associated with it
- Simulators need to ensure events are processed in the increasing order of time
- **Event Queue:** internal queue ordered by simulation time maintained by a simulator



- **Timing wheel:** an efficient data structure to implement even queue
- An array of  $M$  slots indexed by simulation time  $T \% M$

# Processing of Events

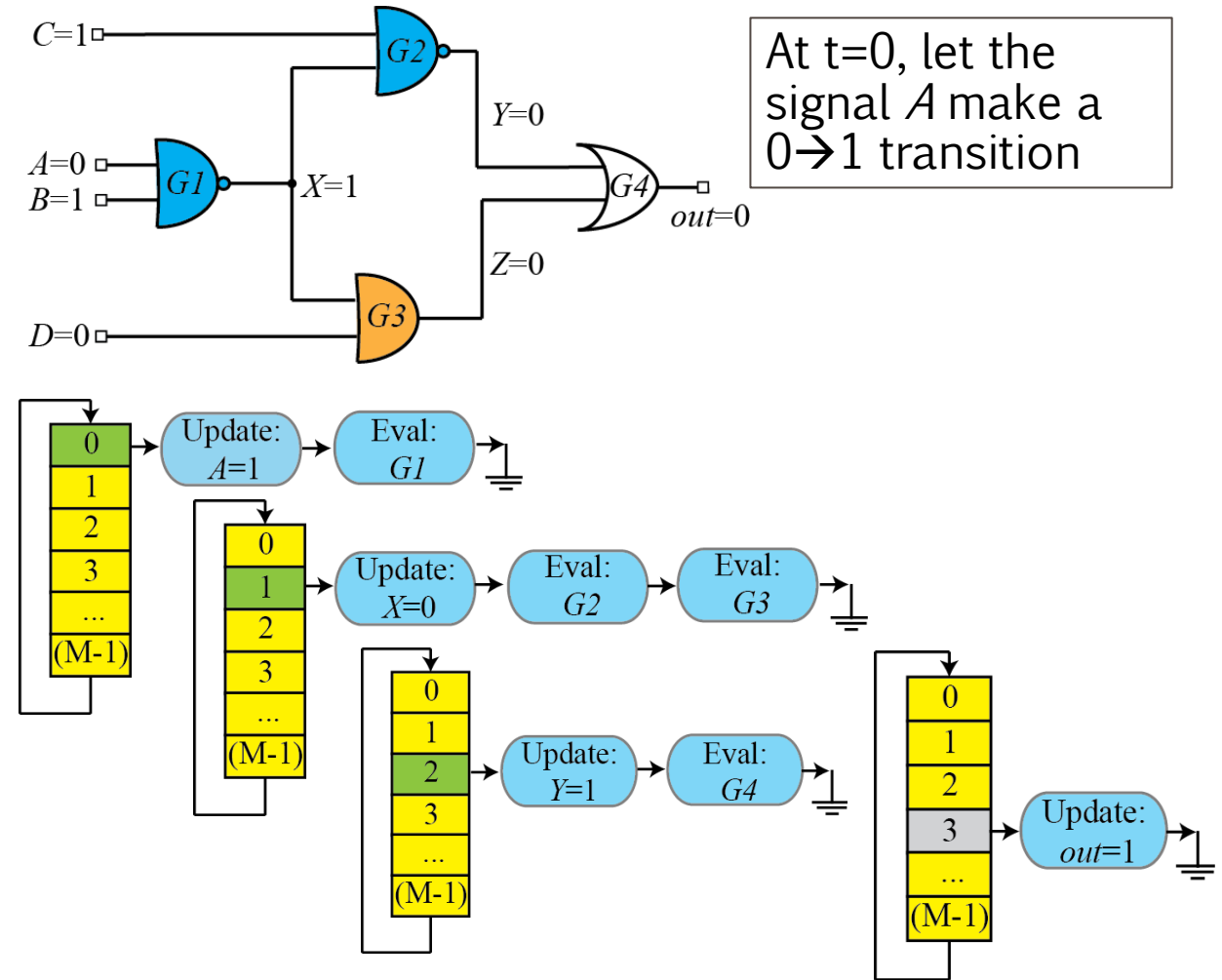
- Applying stimulus creates update events
- Update events can trigger evaluation events
- Evaluation events can further change values of nets/signals and lead to new update events

- Sequence of:

update → evaluate → update →  
evaluate ...

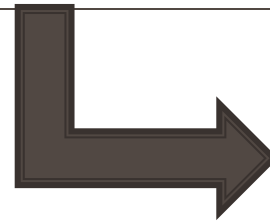
goes on until the end of simulation  
time

Assume that a circuit is as shown  
alongside. Assume that delay of each  
gate is 1 time unit.



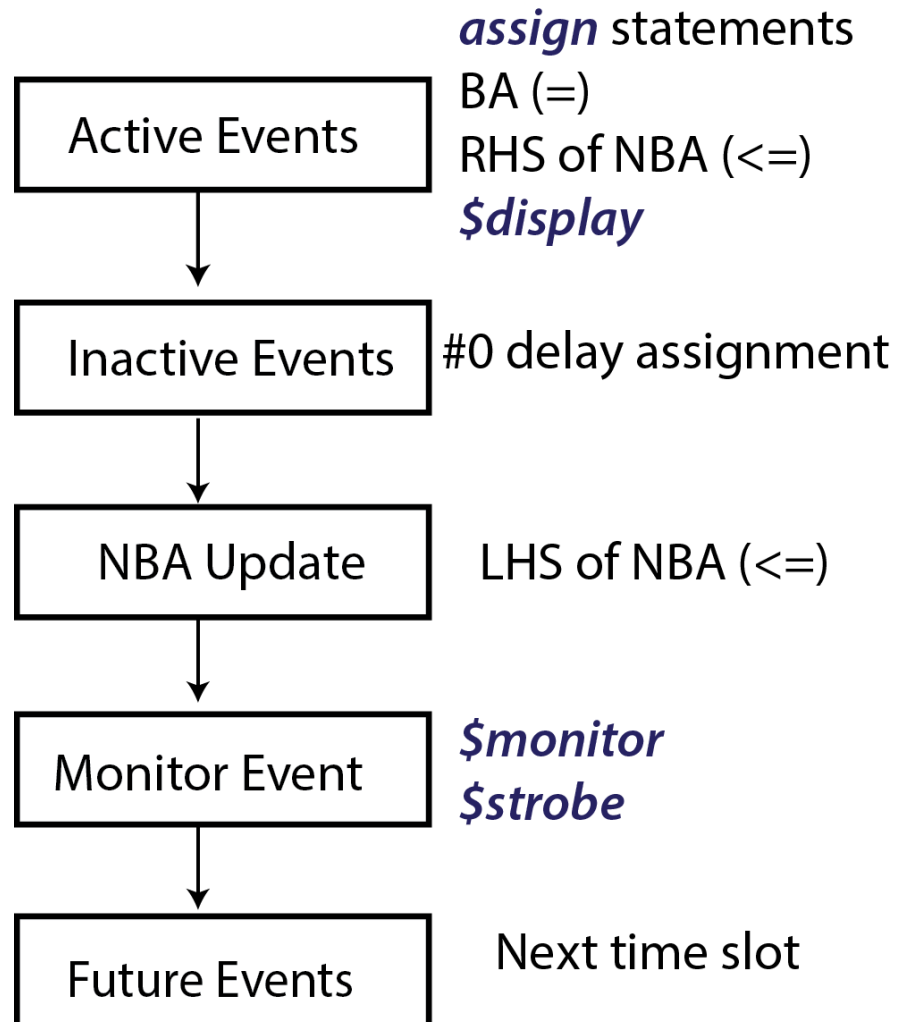
---

# Simulation in Verilog



Mechanism

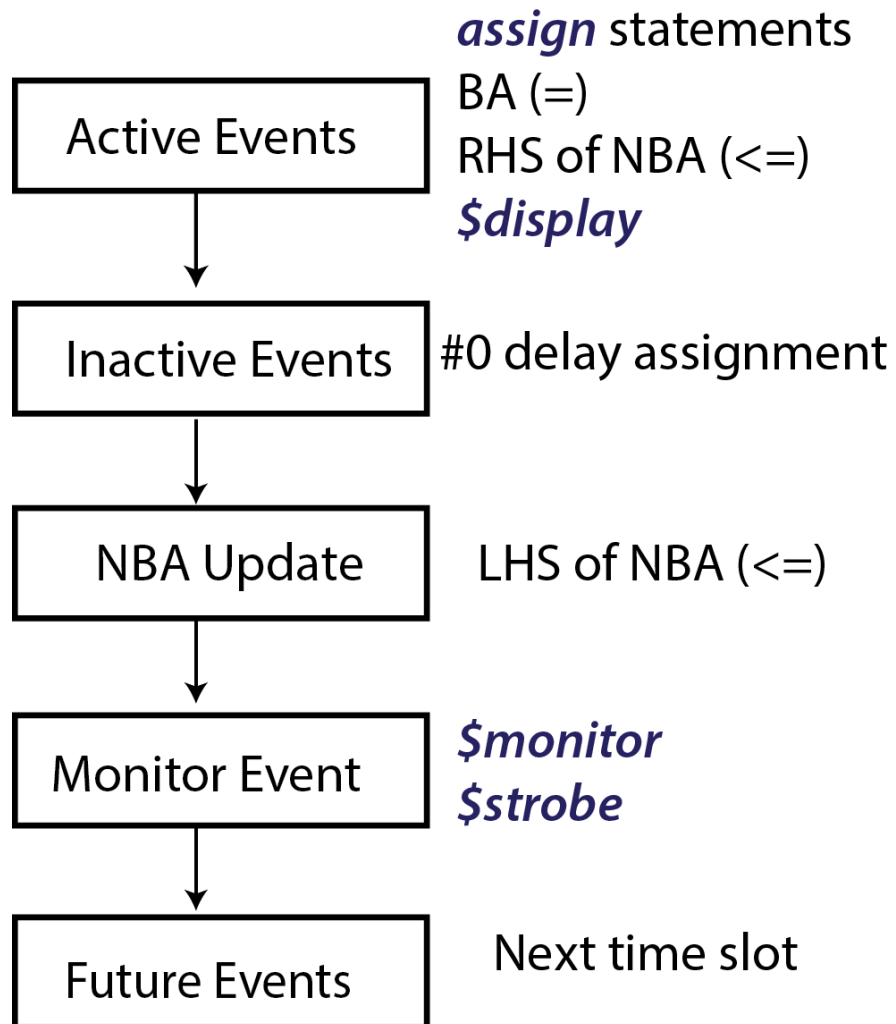
# Mechanism of Simulation : Stratified Queue (1)



## Stratified Verilog Event Queue:

- Conceptual Model to understand how simulation of different Verilog assignments take place
- Events at a given simulation time is divided into five layers and processed top-down
- There is no priority among *Active Events*
  - Execution order among Active Events is arbitrary
  - Source of indeterminism in simulation of Verilog code

# Mechanism of Simulation : Stratified Queue (2)



## Stratified Verilog Event Queue

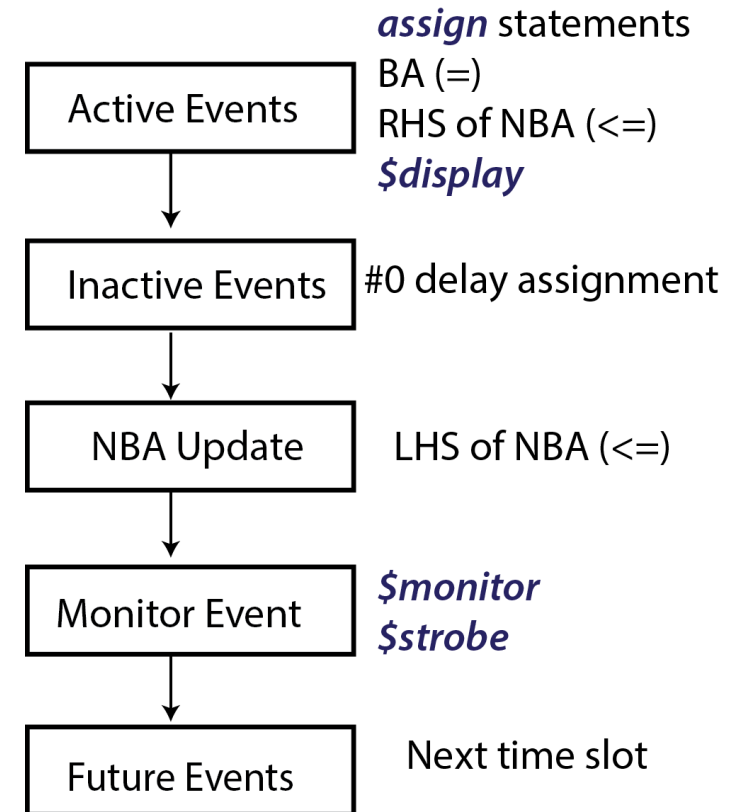
- Statements within a begin-end blocks are executed as they appear in the block: deterministic
- Different blocks can be put in Active Events Queue in any order: source of indeterminism
- NBA Updates can trigger further Active Events

# Mechanism of Simulation: Illustration

For the Verilog code shown along side, let us determine the output of the \$display function.

```
initial begin
    a = 1'b0;
    a <= 1'b1;
    $display("\nValue of a is :%b", a);
end
```

- $a = 1'b0$ ; put in Active Events Queue
- $a \leq 1'b1$ ; RHS put in Active Events Queue and LHS put in NBA Update Queue
- \$display: put in Active Events Queue
- Active Event Queue is processed:
  - $a$  gets value of  $1'b0$
  - \$display produces "0" for  $a$
- NBA Update Queue is processed:
  - $a$  gets value of  $1'b1$



# Race conditions in Verilog : Definition

---

- Verilog language specification (IEEE standard) defines which statements have a guaranteed order of execution and which statements have no guaranteed (indeterminate) order of execution
- **Definition:** When two or more statements that are scheduled to execute in the same simulation time, and would give different results when the order of execution of the statements are changed (as permitted by IEEE standard), then race condition is said to exist
- Race conditions can occur in many situations
- Race conditions are difficult to debug
- Some guidelines or good coding practice can be followed to avoid race conditions

# Race conditions in Verilog : Illustration

---

```
module race();  
  
    reg a, b;  
  
    initial begin  
        a = 0;  
        b = 1;  
    end  
  
    initial begin  
        a = 1;  
        b = 0;  
    end  
  
endmodule
```



# References

---

- S. Saurabh, “Introduction to VLSI Design Flow”. Cambridge: *Cambridge University Press*, 2023.